

AIM

To implement a Random Forest classifier for a binary classification task using the Breast Cancer dataset, evaluate its performance using accuracy, precision, recall, F1-score, confusion matrix and ROC-AUC, and identify the important features using feature importance.

PROCEDURE

1. Import the required Python libraries.
2. Load the Breast Cancer dataset from Scikit-learn.
3. Separate the dataset into input features and target values.
4. Split the dataset into training and testing sets using stratification.
5. Create a Random Forest Classifier.
6. Train the Random Forest model using the training data.
7. Predict the target values for the test data.
8. Calculate accuracy, precision, recall and F1-score.
9. Generate the confusion matrix and classification report.
10. Calculate the ROC-AUC score and plot the ROC curve.
11. Determine the feature importance values from the trained Random Forest model.
12. Display the top 10 important features using a bar chart.
13. Perform hyperparameter tuning using GridSearchCV to obtain the best model parameters.

CODE

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.metrics import f1_score, confusion_matrix, classification_report
from sklearn.metrics import roc_auc_score, roc_curve
```

```
data = load_breast_cancer()
```

```
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="target")
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

```
rf = RandomForestClassifier(
    random_state=42,
    n_jobs=-1
)
```

```
param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [None, 10],
    "min_samples_split": [2, 5],
```

```
"min_samples_leaf": [1, 2]
}
```

```
grid = GridSearchCV(
    rf,
    param_grid,
    scoring="f1",
    cv=3,
    n_jobs=-1
)
```

```
grid.fit(X_train, y_train)
```

```
print("Best Parameters:")
print(grid.best_params_)
```

```
best_rf = grid.best_estimator_
```

```
y_pred = best_rf.predict(X_test)
```

```
y_prob = best_rf.predict_proba(X_test)[:, 1]
```

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_prob)
```

```

cm = confusion_matrix(y_test, y_pred)

print("\nRandom Forest Test Results")
print("-----")
print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall  :", round(recall, 4))
print("F1-Score :", round(f1, 4))
print("ROC-AUC  :", round(auc, 4))

print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=data.target_names,
    yticklabels=data.target_names
)

plt.xlabel("Predicted")

```

```
plt.ylabel("Actual")
plt.title("Random Forest - Confusion Matrix")
plt.show()
```

```
importances = pd.Series(
    best_rf.feature_importances_,
    index=X.columns
)
```

```
top10 = importances.sort_values(
    ascending=False
).head(10)
```

```
plt.figure(figsize=(8, 6))
```

```
top10.sort_values().plot(kind="barh")
```

```
plt.xlabel("Importance")
plt.ylabel("Features")
plt.title("Top 10 Feature Importances")
plt.show()
```

```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
```

```
plt.figure(figsize=(7, 5))
```

```
plt.plot(
    fpr,
```

```

    tpr,
    label=f"Random Forest (AUC = {auc:.3f})"
)

```

```

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--"
)

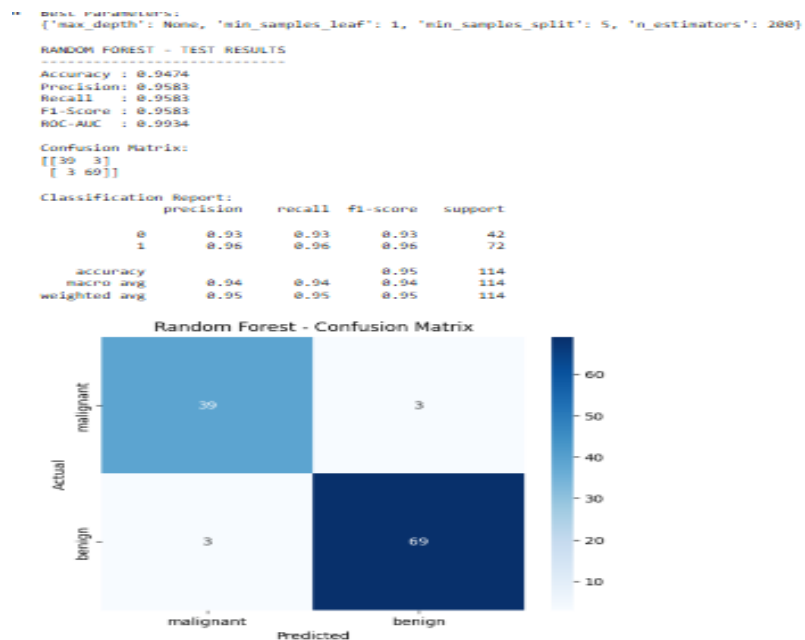
```

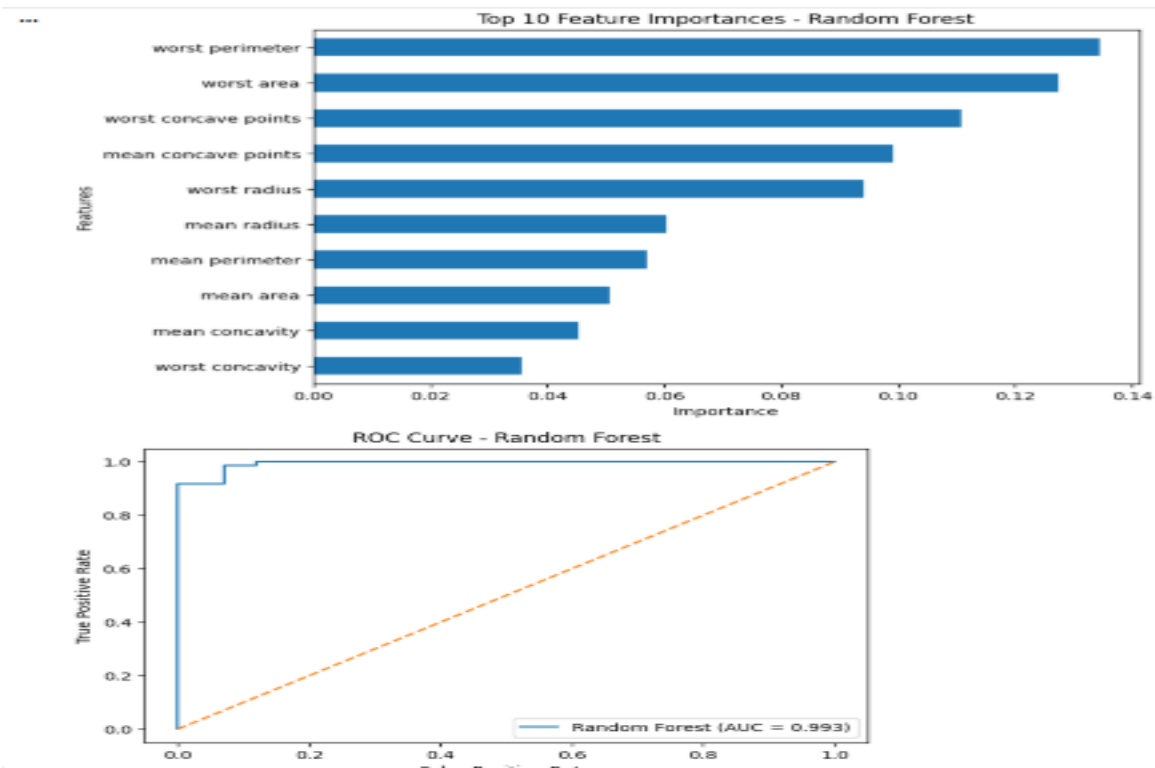
```

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Random Forest")
plt.legend()
plt.show()

```

OUTPUT:





RESULT

The Random Forest classifier was successfully implemented using the Breast Cancer dataset. The model was trained and tested successfully, and its performance was evaluated using accuracy, precision, recall, F1-score, confusion matrix and ROC-AUC. Feature importance was also analyzed to identify the most influential features. Hyperparameter tuning using GridSearchCV was successfully performed to obtain an improved Random Forest model.